

Jira QA Testing with Xray Portfolio Case Study

Sprint 1 Regression Cycle — SauceDemo Web Application

Tester	Karen — ISTQB Foundation Certified
Role target	Quality Engineer
Tools	Jira Cloud + Xray Test Management
Application under test	SauceDemo (saucedemo.com)
Sprint	Sprint 1 — Core Purchase Flow (Login, Cart, Checkout)

This document walks through one full sprint cycle executed in a simulated but fully functional Jira + Xray environment — from sprint planning through defect discovery, developer handoff, and verified resolution. All screenshots are from a live Jira Cloud project.

1. Context

SauceDemo is a demo e-commerce application purpose-built for QA practice, containing several test accounts with distinct, intentional behaviors (a valid user, a locked-out user, and accounts with UI and backend quirks). For this case study, I set up a complete Jira Cloud project (**TestingPortfolio**) with Xray Test Management to simulate a real manual QA workflow: sprint planning, test design, execution, defect reporting, and developer handoff.

Sprint 1 goal: Validate the core purchase flow (login → add to cart → checkout) ahead of release, and confirm that known edge cases (locked-out accounts) are handled gracefully.

Scope

- User authentication (TES-1) — 3 story points
- Add to Cart functionality (TES-3) — 3 story points
- Checkout flow (TES-5) — 5 story points

Out of scope: product sorting, catalog display, and menu navigation — deferred to Sprint 2.

Spaces / TestingPortfolio

TES board

Summary Timeline Backlog Active sprints Calendar Capacity Reports List Forms Goals Components Development Code More

Search backlog

Version Epic Type Label Quick filters

Recently Updated Assigned to me

TES Sprint 1 26 Aug – 9 Sep (4 work items)

0 8 3 Complete sprint

- TES-1 Validate login functionality across all test user accounts Done 3
- TES-5 Validate full checkout flow (Your Information → Overview → Complete) In Progress 5
- TES-3 Validate 'Add to Cart' and cart badge counter functionality In QA 3
- TES-15 Sprint 1 - QA Plan In Progress 1

Backlog (7 work items)

0 0 0 Create sprint

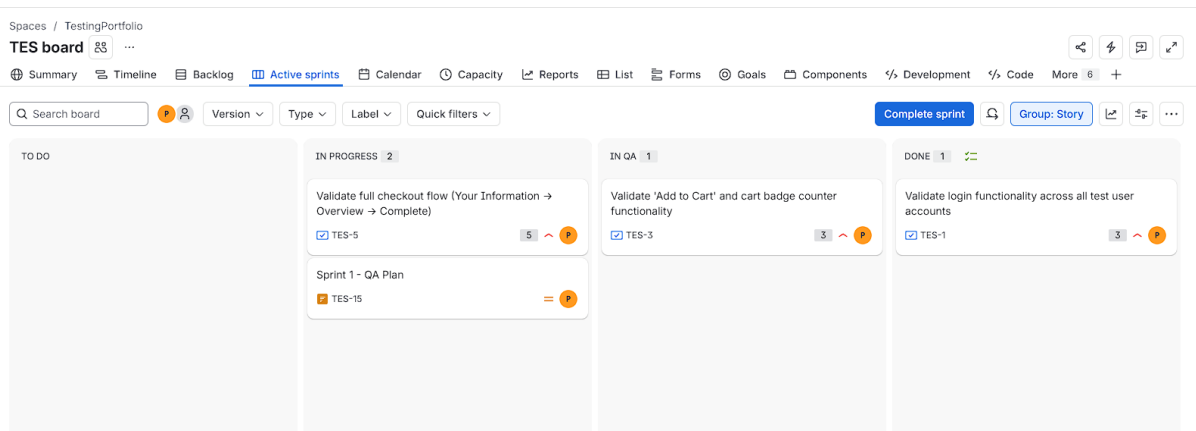
- TES-7 Checkout: 'error_user' cannot complete checkout, no error message shown To Do 1
- TES-2 Verify product catalog displays correctly for standard user To Do 1
- TES-4 Verify product sorting options (Name A-Z, Z-A, Price low-high, high-low) To Do 1
- TES-6 Verify hamburger menu options (Logout, Reset App State, About) To Do 1
- TES-9 Performance: 'performance_glitch_user' has ~5s delay on login To Do 1
- TES-8 Catalog: product images broken for 'problem_user' To Do 1
- TES-16 Sprint 1 - Execution Cycle 1 To Do 1

+ Create

Sprint 1 backlog — 3 stories committed, story points assigned, ranked by priority.

2. Process

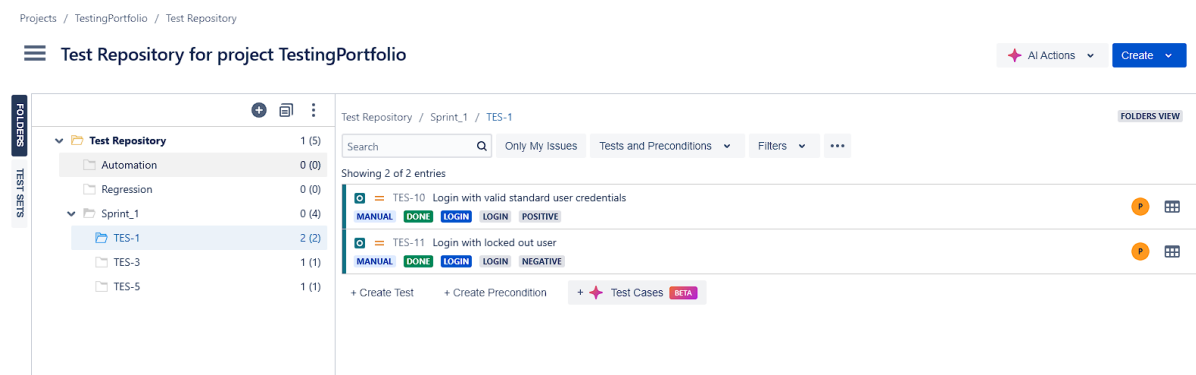
Rather than mixing Test issues into the development board (a common anti-pattern), Tests were kept in Xray's dedicated Test Repository, organized in folders that mirror the sprint structure (*Sprint_1 / TES-1*, *Sprint_1 / TES-3*, etc.). A custom workflow state, **In QA**, was added between *In Progress* and *Done* to make the dev → QA handoff explicit and trackable — including a **Reopen** transition back to development when defects are found.



Active sprint board with the custom To Do / In Progress / In QA / Done workflow.

Test Organization

Tests were organized in Xray's Test Repository using a sprint-based folder structure (mirroring the convention used in production QA teams), with each story's tests nested under its own subfolder for traceability. Two structural folders were also set up at the root level: **Automation** and **Regression**. Both are intentionally empty in this manual-only exercise — in a real pipeline, *Automation* would be populated automatically via CI/CD (e.g. Selenium/Playwright/Cypress results synced through the Xray API rather than entered by hand), and *Regression* would accumulate reusable tests re-run across releases as the product matures.

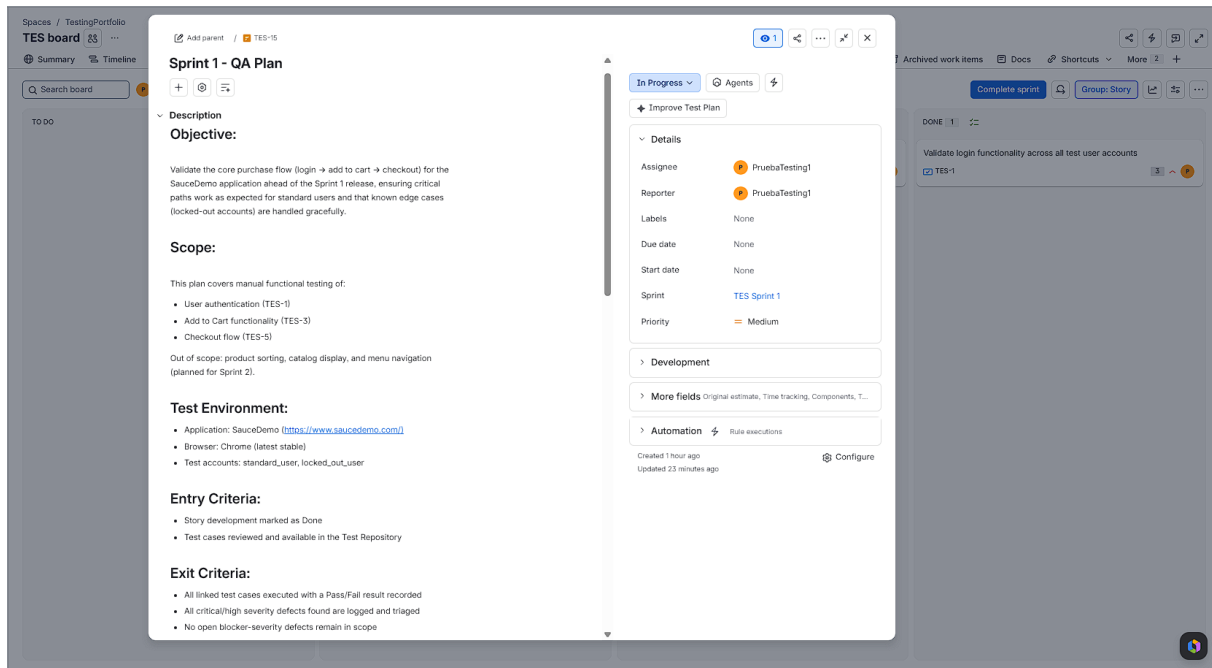


Test Repository structure: Sprint-based folders with per-story test grouping.

Test Planning

A dedicated **Test Plan** issue (TES-15) was created with a documented objective, scope, test environment, and entry/exit criteria — the same structure used in professional QA documentation — grouping the 4 tests

relevant to this sprint's stories.



Sprint 1 - QA Plan

Description

Objective:

Validate the core purchase flow (login → add to cart → checkout) for the SauceDemo application ahead of the Sprint 1 release, ensuring critical paths work as expected for standard users and that known edge cases (locked-out accounts) are handled gracefully.

Scope:

This plan covers manual functional testing of:

- User authentication (TES-1)
- Add to Cart functionality (TES-3)
- Checkout flow (TES-5)

Out of scope: product sorting, catalog display, and menu navigation (planned for Sprint 2).

Test Environment:

- Application: SauceDemo (<https://www.saucedemo.com/>)
- Browser: Chrome (latest stable)
- Test accounts: standard_user, locked_out_user

Entry Criteria:

- Story development marked as Done
- Test cases reviewed and available in the Test Repository

Exit Criteria:

- All linked test cases executed with a Pass/Fail result recorded
- All critical/high severity defects found are logged and triaged
- No open blocker-severity defects remain in scope

Details

- Assignee: PruebaTesting1
- Reporter: PruebaTesting1
- Labels: None
- Due date: None
- Start date: None
- Sprint: TES Sprint 1
- Priority: Medium

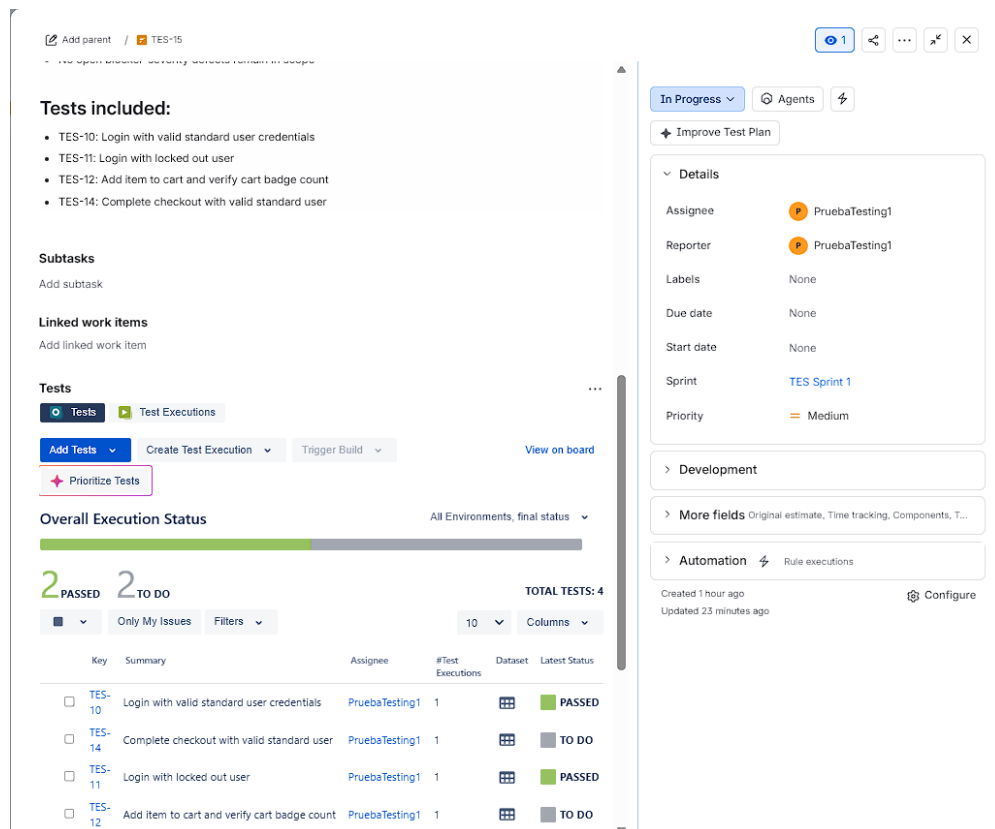
Development

More fields Original estimate, Time tracking, Components, T...

Automation Rule executions

Created 1 hour ago
Updated 23 minutes ago

Test Plan documentation: objective, scope, environment, and entry/exit criteria.



Tests included:

- TES-10: Login with valid standard user credentials
- TES-11: Login with locked out user
- TES-12: Add item to cart and verify cart badge count
- TES-14: Complete checkout with valid standard user

Subtasks

Add subtask

Linked work items

Add linked work item

Tests

Tests | Test Executions

Add Tests | Create Test Execution | Trigger Build | View on board

Prioritize Tests

Overall Execution Status

All Environments, final status

2 PASSED 2 TO DO

TOTAL TESTS: 4

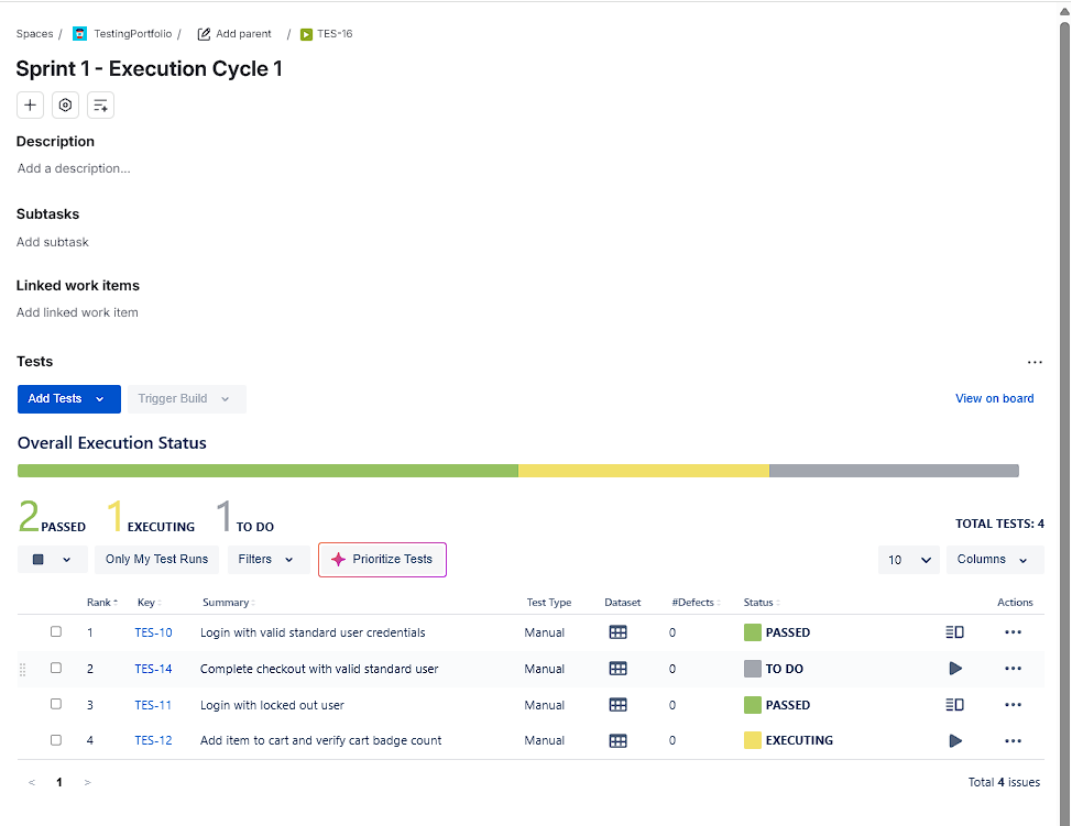
Only My Issues | Filters | 10 | Columns

Key	Summary	Assignee	#Test Executions	Dataset	Latest Status
☐ TES-10	Login with valid standard user credentials	PruebaTesting1	1		PASSED
☐ TES-14	Complete checkout with valid standard user	PruebaTesting1	1		TO DO
☐ TES-11	Login with locked out user	PruebaTesting1	1		PASSED
☐ TES-12	Add item to cart and verify cart badge count	PruebaTesting1	1		TO DO

Test Plan (continued): linked tests and real-time overall execution status.

Test Execution

Tests were executed manually against the live application and results were logged step by step in a Test Execution cycle, with Pass/Fail tracked per action — not just per test.



Sprint 1 — Execution Cycle 1: real-time Pass / Executing / To Do status per test.

3. Finding

While executing **TES-14 (Complete checkout with valid standard user)**, I extended the test beyond its written script and ran it against **error_user** — an account known to have unstable behavior — to explore edge cases beyond the happy path.

The checkout form and totals calculated correctly through the Overview step, but clicking **“Finish”** produced no navigation and no error feedback of any kind — the order was never confirmed. I also inspected the browser console during the flow and identified a secondary, lower-severity issue: repeated failed background analytics calls (401 Unauthorized) unrelated to the main defect.

The screenshot displays the Xray Test Execution interface for a test titled "TES-14 (Complete checkout with valid standard user)". The test is marked as "FAILED". The interface shows the test details, including the test issue links, steps, and activity. The test is associated with the repository "Test Repository / Sprint_1 / TES-5".

Test details

Test Issue Links

- TES-5 Validate full checkout flow (Your information -> Overview -> Complete)

Steps

Step	Action	Data	Expected Result	Step State
1	Log in as standard_user, add 1 product to cart, go to Cart, click Checkout	-	Checkout: Your information page is displayed	PASSED
2	Fill in First Name, Last Name, Zip and click Continue	First Name: Jane / Last Name: Doe / Zip: 12345	Checkout Overview page displays item, tax and total correctly	FAILED
3	Click "Finish"	-	Order confirmation page is displayed with message "Thank you for your order"	FAILED

Activity

Step-level failure recorded directly in the Xray Test Execution.

The screenshot shows a web browser window with the URL `https://www.saucedemo.com/checkout-step-two.html`. The page is titled "Swag Labs" and displays a checkout overview. The main content area shows a table with one item: "Sauce Labs Backpack" with a price of \$29.99. Below the table, there are sections for "Payment Information" (SauceCard #31337), "Shipping Information" (Free Pony Express Delivery!), and "Price Total" (Item total: \$29.99, Tax: \$2.40, Total: \$32.39). At the bottom of the page is a "Cancel" button.

The Chrome DevTools Network panel is open, showing a list of requests. The first request is a GET request to `favicon.ico`. The second request is a POST request to `https://events.backtrace.io/api/summed-events/submit?universe=UNIVERSE&to=401` with a status of 401 (Unauthorized). The third request is a POST request to `https://events.backtrace.io/api/unique-events/submit?universe=UNIVERSE&to=401` with a status of 401 (Unauthorized). The fourth request is a POST request to `https://events.backtrace.io/api/summed-events/submit?universe=UNIVERSE&to=401` with a status of 401 (Unauthorized). The fifth request is a POST request to `https://events.backtrace.io/api/unique-events/submit?universe=UNIVERSE&to=401` with a status of 401 (Unauthorized).

The Console panel shows a message: "Following Content Security Policy directive 'script-src 'self' 'wasm-unsafe-eval' 'inline-speculation-rules' chrome-extension://cf312404-8d30-463f-a913-1be15294e9e9/'. Either the 'unsafe-inline' keyword, a hash ('sha256-+osafquB23g9K0XV1s9WlF6X2y8DNB1M4W8Bj/zdc='), or a nonce ('nonce-...') is required to enable inline execution. The action has been blocked."

Browser DevTools evidence — confirms the app itself functioned correctly up to this point; the failure is isolated to the Finish action.

Defect Report

The defect was logged with precise reproduction steps, severity/priority classification, and both video and screenshot evidence attached — then linked to the failing test step for full traceability.

Spaces / TestingPortfolio / Add parent / TES-7

Checkout: 'error_user' cannot complete checkout, no error message shown



Key details

Description

Environment: Chrome 127 (Brave), Windows 11

Steps to Reproduce:

- 1. Log in as error_user / secret_sauce
- 2. Add any product to cart, go to Checkout
- 3. Fill in valid First Name, Last Name, Zip and click 'Continue' (this step works correctly, advances to Overview)
- 4. Verify totals on the Overview page (also correct)
- 5. Click 'Finish'

Expected Result: User is redirected to the order confirmation page with the message "Thank you for your order!"

Actual Result: Nothing happens — the app does not navigate anywhere and shows no error message or feedback. The order is never confirmed.

Severity: High (blocks core purchase completion for this user type, with zero user-facing feedback)

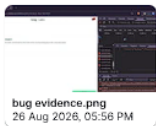
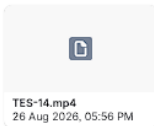
Priority: High

Environment

None

Attachments 2

... +



Subtasks

Add subtask

Linked work items

+

created by

TES-16 Sprint 1 - Execution Cycle 1	In Progress	
TES-14 Complete checkout with valid standard user	In Progress	

Bug TES-7 — final report with corrected repro steps and attached evidence (video + screenshot).

4. Resolution

The parent story (TES-5) was returned to **In Progress** using the custom *Reopen* transition, with the defect linked directly on the story for visibility — the story could not be marked complete while a blocking defect remained open.

The screenshot shows the 'Test details' page for a manual test. It includes a 'Test Issue Links' section with two links: 'TES-5 Validate full checkout flow (Your Information -> Overview -> Complete)' and 'TES-7 Checkout: 'error_user' cannot complete checkout, no error message shown'. The 'Steps' section contains three steps. Step 1 is 'PASSED', Step 2 is 'PASSED', and Step 3 is 'FAILED'. A tooltip for Step 3 shows the defect 'TES-7 Checkout: 'error_user' cannot complete checkout - IN PROGRESS'.

Step	Action	Data	Expected Result	Step State
1	Log in as standard_user, add 1 product to cart, go to Cart, click Checkout	-	Checkout: Your information page is displayed	PASSED
2	Fill in First Name, Last Name, Zip and click Continue	First Name: Jane / Last Name: Doe / Zip: 12345	Checkout Overview page displays item, tax and total correctly	PASSED
3	Click 'Finish'	-	Order confirmation page is displayed with message 'Thank you for your order!'	FAILED

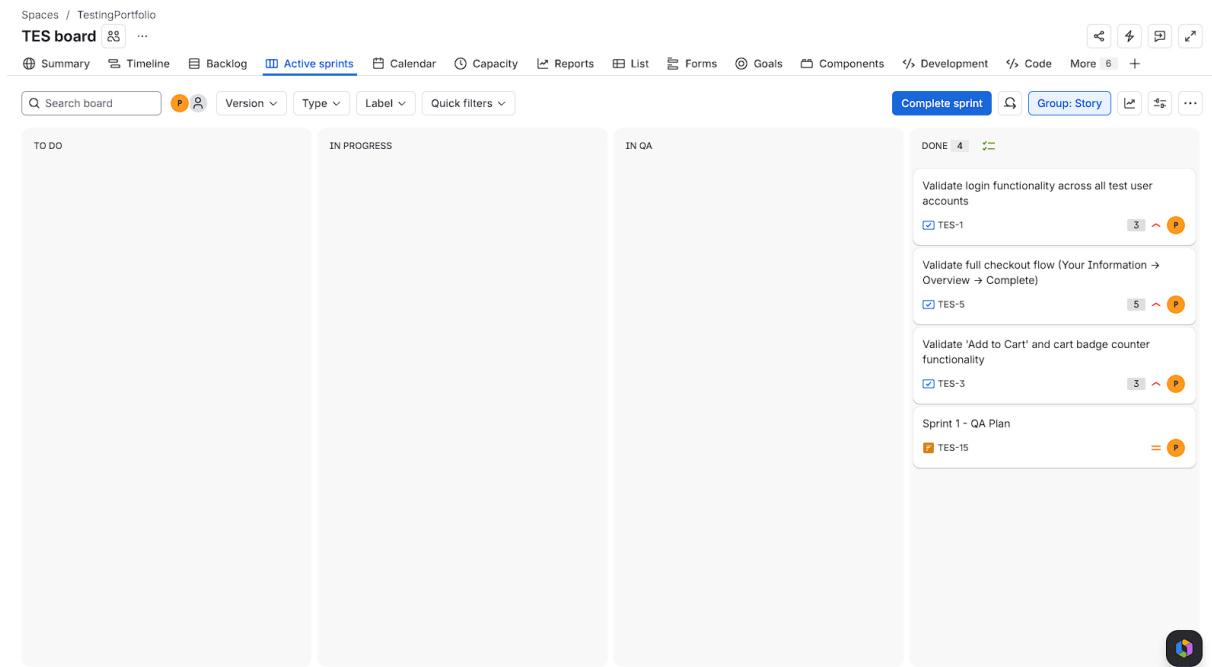
TES-5 reopened and linked to the active defect TES-7, driven from the failed test step.

Verification

Once the fix was confirmed, a targeted retest was run against the previously failing test only (rather than re-running the full suite) — a deliberate, time-efficient regression strategy — before closing the defect and approving the story.

The screenshot shows the 'TestingPortfolio' interface. It includes a navigation bar with tabs like 'Summary', 'Timeline', 'Backlog', 'Active sprints', 'Calendar', 'Capacity', 'Reports', 'List', 'Forms', 'Goals', 'Components', and 'Development'. The 'Testing Board' tab is active. The main content area shows 'Test Executions for Test Plan Sprint 1 - QA Plan'. It lists two test executions: 'TES-17 Test Execution for Test Plan TES-15' and 'TES-16 Sprint 1 - Execution Cycle 1'. The 'TES-17' execution is marked as 'TO DO' and 'TES-15: SPRINT 1 - QA PLAN'. The 'TES-16' execution is marked as 'IN PROGRESS' and 'TES-15: SPRINT 1 - QA PLAN'.

Retest execution confirming the fix, run independently from the original execution cycle.



Sprint 1 closed: all committed stories verified and moved to Done.

Outcome

- 3/3 committed stories (11 story points) delivered and QA-approved
- 1 high-severity defect found, reported with full evidence, and verified fixed
- 1 additional low-severity defect identified through exploratory testing (browser console inspection)
- Full traceability maintained: Story → Test → Execution → Defect → Resolution